

Oráculos y primeros algoritmos cuánticos

¿Cómo calculamos una función booleana?

Considere una función Booleana $f : \{0, 1\}^n \longrightarrow \{0, 1\}$

Queremos utilizar f dentro de un circuito cuántico

Podríamos intentar representar su evaluación mediante la transformación

$$|x\rangle \longmapsto |f(x)\rangle$$

¿Cómo calculamos una función booleana?

El problema es que f no es una función inyectiva si $n \geq 2$

- Podemos entonces perder información y la computación no sería reversible

Nos gustaría representar f a través de una matriz unitaria U_f

- ¿Cómo podemos calcular $f(x)$ conservando la información de entrada?

El oráculo XOR

El **oráculo XOR** U_f se define de la siguiente forma, suponiendo que $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

$$U_f|yx\rangle = |(y \oplus f(x))x\rangle$$

En particular, si inicializamos el qubit y en $|0\rangle$, tenemos que:

$$U_f|0x\rangle = |f(x)x\rangle$$

¿Existe este oráculo para cualquier función booleana?

Sabemos que U_f es una matriz de $2^{n+1} \times 2^{n+1}$

Vamos a demostrar que U_f es una matriz unitaria

- Para esto, vamos a demostrar primero que $U_f U_f = I$, donde I es la matriz identidad de $2^{n+1} \times 2^{n+1}$

Consideramos un vector arbitrario $|\psi\rangle \in \mathbb{C}^{2^{n+1}}$. Vamos a demostrar que $U_f U_f |\psi\rangle = |\psi\rangle$

- De esto se concluye que $U_f U_f = I$

¿Existe este oráculo para cualquier función booleana?

Sabemos que:

$$|\psi\rangle = \sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |yx\rangle$$

Por lo tanto:

$$\begin{aligned} U_f U_f |\psi\rangle &= U_f U_f \left(\sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |yx\rangle \right) \\ &= U_f \left(\sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} U_f |yx\rangle \right) \\ &= U_f \left(\sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |(y \oplus f(x))x\rangle \right) \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

$$\begin{aligned} &= U_f \left(\sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |(y \oplus f(x))x\rangle \right) \\ &= \sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} U_f |(y \oplus f(x))x\rangle \\ &= \sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |(y \oplus f(x) \oplus f(x))x\rangle \\ &= \sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |yx\rangle \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Tenemos que: $U_f U_f = I$

Vamos a demostrar que $U_f^\dagger = U_f$

- De esto concluimos que U_f es una matriz unitaria puesto que $U_f^\dagger U_f = U_f U_f = I$

¿Existe este oráculo para cualquier función booleana?

Considere la siguiente matriz, donde $X^0 = I$ y $X^1 = X$:

$$\sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u|$$

Observe que $X^{f(x)} |y\rangle = |y \oplus f(x)\rangle$ para $x \in \{0,1\}^n$ e $y \in \{0,1\}$

¿Existe este oráculo para cualquier función booleana?

Entonces tenemos lo siguiente para $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

$$\begin{aligned} \left(\sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u| \right) (|y\rangle \otimes |x\rangle) &= \sum_{u \in \{0,1\}^n} (X^{f(u)} \otimes |u\rangle\langle u|)(|y\rangle \otimes |x\rangle) \\ &= \sum_{u \in \{0,1\}^n} X^{f(u)} |y\rangle \otimes |u\rangle\langle u|x\rangle \\ &= \sum_{u \in \{0,1\}^n} X^{f(u)} |y\rangle \otimes |u\rangle\langle u | x\rangle \\ &= X^{f(x)} |y\rangle \otimes |x\rangle = |y \oplus f(x)\rangle \otimes |x\rangle = U_f(|y\rangle \otimes |x\rangle) \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Concluimos que:
$$U_f = \sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u|$$

Por lo tanto:

$$\begin{aligned} U_f^\dagger &= \left(\sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u| \right)^\dagger \\ &= \sum_{u \in \{0,1\}^n} (X^{f(u)} \otimes |u\rangle\langle u|)^\dagger \\ &= \sum_{u \in \{0,1\}^n} (X^{f(u)})^\dagger \otimes (|u\rangle\langle u|)^\dagger = \sum_{u \in \{0,1\}^n} X^{f(u)} \otimes |u\rangle\langle u| = U_f \end{aligned}$$

¿Existe este oráculo para cualquier función booleana?

Tenemos entonces que la matriz U_f es unitaria

Concluimos que toda función booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$ admite un oráculo XOR unitario

- f admite un oráculo que puede ser implementado
- ¿Cómo construimos este operador usando las compuertas que conocemos?

Consultar una función

En una consulta clásica elegimos una entrada $x \in \{0, 1\}^n$ y recibimos $f(x)$

En una consulta cuántica elegimos $|\psi\rangle = \sum_{y \in \{0,1\}} \sum_{x \in \{0,1\}^n} \alpha_{yx} |yx\rangle$ y
recibimos $U_f |\psi\rangle$

En el análisis del costo de un algoritmo consideramos el número de veces que se llama a este oráculo

Un oráculo XOR para AND

Considere $f(x_1, x_0) = x_1 \wedge x_0$

Queremos construir el oráculo $|yx_1x_0\rangle \mapsto |(y \oplus (x_1 \wedge x_0))x_1x_0\rangle$

¿Cómo hacemos esto?

- Usamos una compuerta de Toffoli

Un oráculo XOR para NOT

Considere $f(x_0) = \neg x_0$

Queremos construir el oráculo $|yx_0\rangle \mapsto |(y \oplus \neg x_0)x_0\rangle$

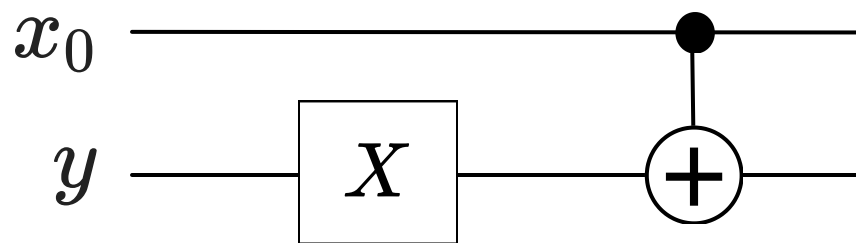
¿Cómo hacemos esto?

- Podemos usar la equivalencia: $y \oplus \neg x_0 \equiv y \oplus (1 \oplus x_0) \equiv (y \oplus 1) \oplus x_0$
- ¿Qué compuertas necesitamos para implementar esto?

Un oráculo XOR para NOT

Considere $f(x_0) = \neg x_0$

Queremos construir el oráculo $|yx_0\rangle \mapsto |(y \oplus \neg x_0)x_0\rangle$



Un oráculo XOR para OR

Considere $f(x_1, x_0) = x_1 \vee x_0$

Queremos construir el oráculo $|yx_1x_0\rangle \longmapsto |(y \oplus (x_1 \vee x_0))x_1x_0\rangle$

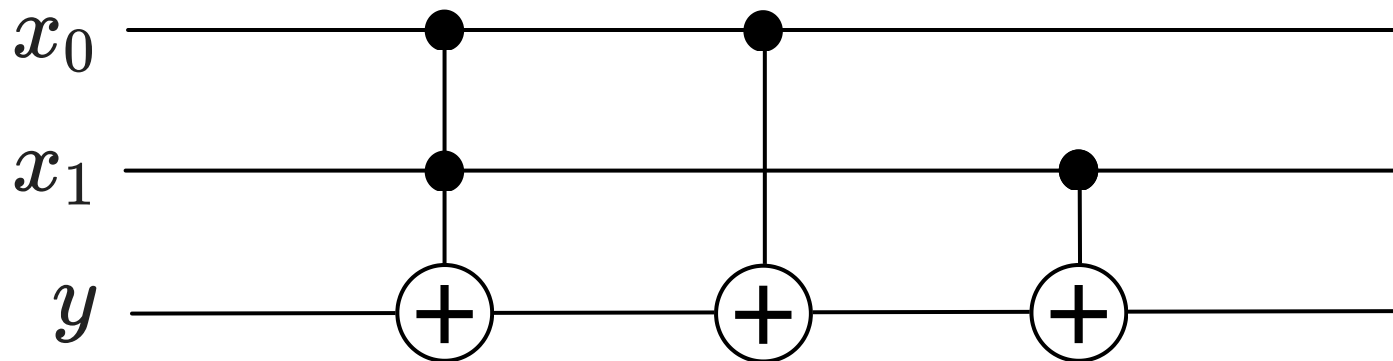
¿Cómo hacemos esto?

- Podemos usar la equivalencia: $x_1 \vee x_0 \equiv x_1 \oplus x_0 \oplus (x_1 \wedge x_0)$
- ¿Qué compuertas necesitamos para implementar esto?

Un oráculo XOR para OR

Considere $f(x_1, x_0) = x_1 \vee x_0$

Queremos construir el oráculo $|yx_1x_0\rangle \mapsto |(y \oplus x_1 \oplus x_0 \oplus (x_1 \wedge x_0))x_1x_0\rangle$



Un oráculo XOR para AND generalizado

Considere $f(x_3, x_2, x_1, x_0) = x_3 \wedge x_2 \wedge x_1 \wedge x_0$

Ejercicio: construya un oráculo XOR para esta función

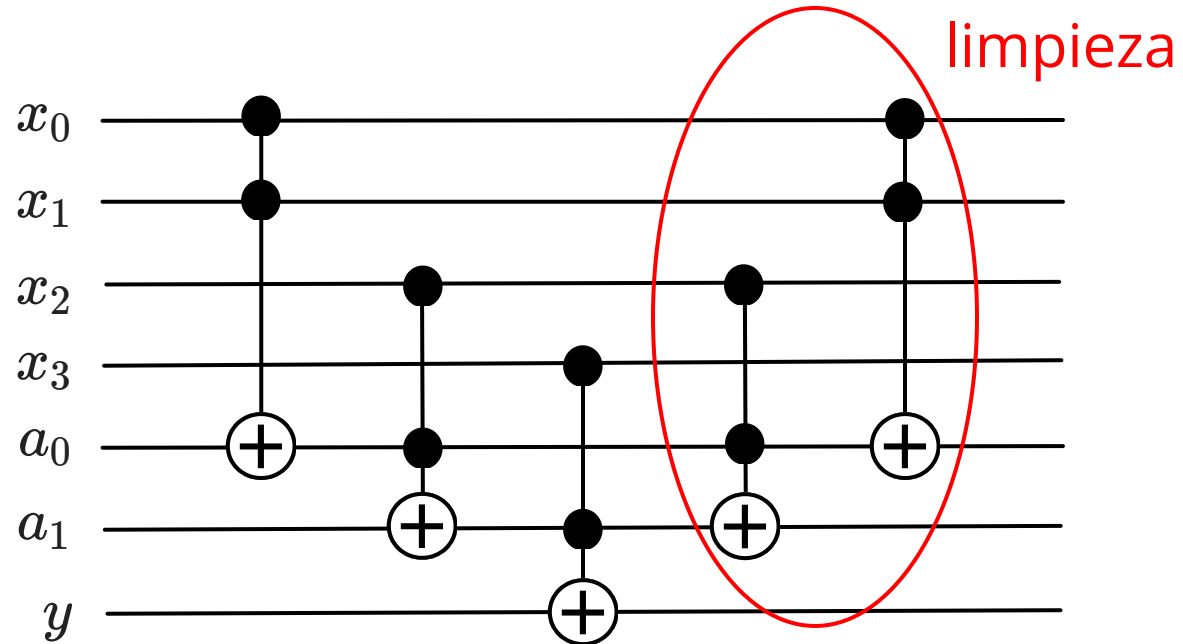
Solución:

Un oráculo XOR para AND generalizado

Considere $f(x_3, x_2, x_1, x_0) = x_3 \wedge x_2 \wedge x_1 \wedge x_0$

Ejercicio: construya un oráculo XOR para esta función

Solución:



Un oráculo XOR para AND generalizado

En este caso el oráculo tiene dos qubits auxiliares: a_1 y a_0

- El oráculo realiza entonces la siguiente transformación considerando el valor 0 en los qubits auxiliares:

$$|y00x_3x_2x_1x_0\rangle \longmapsto |(y \oplus (x_3 \wedge x_2 \wedge x_1 \wedge x_0))00x_3x_2x_1x_0\rangle$$

Ejercicio: construya un oráculo XOR para la función

$$f(x_{n-1}, \dots, x_0) = \bigwedge_{i=0}^{n-1} x_i$$

Un oráculo XOR que reconoce una entrada

Para una cadena fija $a \in \{0, 1\}^n$, definimos

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Su oráculo debe realizar $U_{m_a}|yx\rangle = |(y \oplus m_a(x))x\rangle$

Ejercicio: Construya un oráculo para $a = (1, 1, 0)$

Un oráculo XOR que reconoce una entrada

Para una cadena fija $a \in \{0, 1\}^n$, definimos

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Su oráculo debe realizar $U_{m_a}|yx\rangle = |(y \oplus m_a(x))x\rangle$

En este caso tenemos que $m_a(x_2, x_1, x_0) = x_2 \wedge x_1 \wedge \neg x_0$

Solución:

Vamos a construir un oráculo que realiza lo siguiente:

$$|y0x_2x_1x_0\rangle \longmapsto |(y \oplus m_a(x_2, x_1, x_0))0x_2x_1x_0\rangle$$

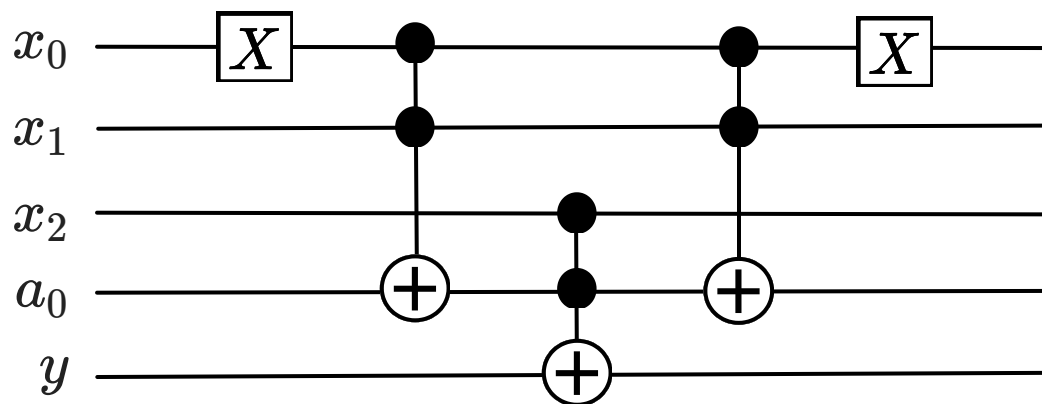
Un oráculo XOR que reconoce una entrada

Para una cadena fija $a \in \{0, 1\}^n$, definimos

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Su oráculo debe realizar $U_{m_a}|yx\rangle = |(y \oplus m_a(x))x\rangle$

Solución:



Construcción para una función booleana arbitraria

Para una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ tenemos que:

$$f(x) = \bigoplus_{a \in \{0,1\}^n : f(a)=1} m_a(x)$$

Ejercicio: Explique cómo esta representación puede ser utilizada para construir U_f , considerando que ya sabemos construir U_{m_a}

Solución del ejercicio en Qiskit

Vamos a mostrar cómo la siguiente representación puede ser utilizada para construir U_f para una función Booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$:

$$f(x) = \bigoplus_{a \in \{0,1\}^n : f(a)=1} m_a(x)$$

Para la construcción en Qiskit de U_f suponemos que $n \geq 3$

Solución del ejercicio en Qiskit

```
from qiskit import QuantumCircuit
import numpy as np

def circuito_entrada(entrada):
    n = len(entrada)
    circuito = QuantumCircuit(2*n-1)
    for i in range(n):
        if entrada[i] == 0:
            circuito.x(n-1-i)
    circuito.ccx(0,1,n)
    for i in range(2,n):
        circuito.ccx(i,i+n-2,i+n-1)
    for i in range(n-2,1,-1):
        circuito.ccx(i,i+n-2,i+n-1)
    circuito.ccx(0,1,n)
    for i in range(n):
        if entrada[i] == 0:
            circuito.x(n-i-1)
    return(circuito)
```

Recuerde que el orden de los bits de entrada desde más a menos significativo es `entrada[0]`, `entrada[1]`, ..., `entrada[n-1]`

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-1)  
    for i in range(len(tabla)):  
        circuito = circuito.compose(circuito_entrada(tabla[i]))  
        if i < len(tabla)-1:  
            circuito.barrier()  
    return circuito
```

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-  
    for i in range(len(tabla)):  
        circuito = circuito.compose(  
            if i < len(tabla)-1:  
                circuito.barrier()  
    return circuito
```

Contiene las entradas
donde la función
Booleana vale 1,
sin repeticiones

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-1)  
    for i in range(len(tabla)):  
        circuito = circuito.compose(circuito_entrada(tabla[i]))  
        if i < len(tabla)-1:  
            circuito.barrier()  
    return circuito
```

Separa las implementaciones
para distintas entradas

Queremos aplicar la implementación a una función Booleana $f(x_3, x_2, x_1, x_0)$ que es igual a 1 exactamente en las tuplas:

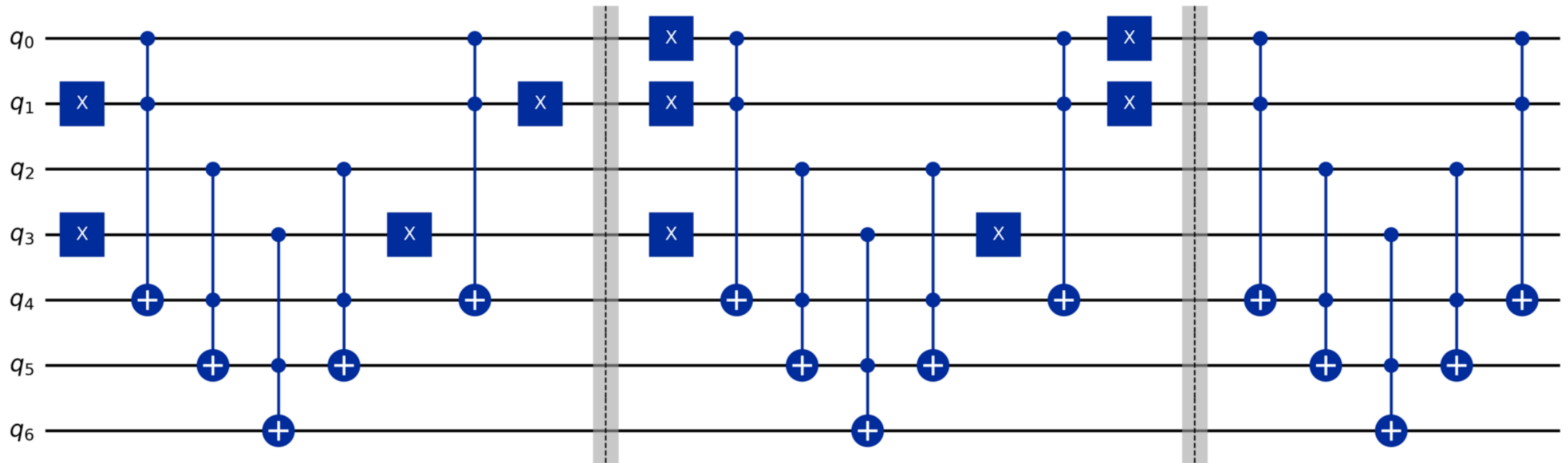
$(0, 1, 0, 1), (0, 1, 0, 0), (1, 1, 1, 1)$

Solución del ejercicio en Qiskit

```
def circuito_tabla(tabla, dim):  
    circuito = QuantumCircuit(2*dim-1)  
    for i in range(len(tabla)):  
        circuito = circuito.compose(circuito_entrada(tabla[i]))  
        if i < len(tabla)-1:  
            circuito.barrier()  
    return circuito
```

```
tabla = [ [0,1,0,1], [0,1,0,0], [1, 1, 1, 1] ]  
dim = 4  
circuito = circuito_tabla(tabla, dim)  
circuito.draw("mpl")
```

Solución del ejercicio en Qiskit



Construcción reversible a partir de un circuito clásico

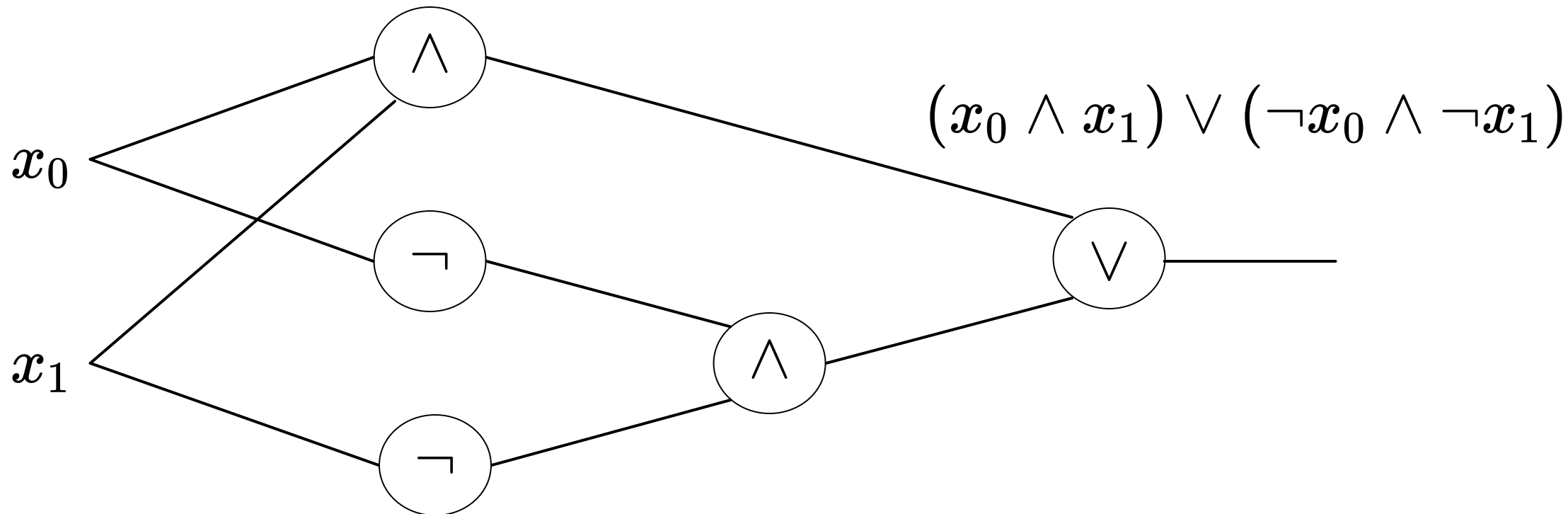
La construcción de U_f a partir de la tabla de verdad de f toma tiempo exponencial en el peor caso

Una mejor alternativa es construir U_f a partir de un circuito clásico C_f para la función f

- Suponemos que este circuito usa compuertas $\neg$, $\wedge$ y $\vee$
- En muchos casos tenemos un circuito clásico de tamaño pequeño para especificar una función Booleana

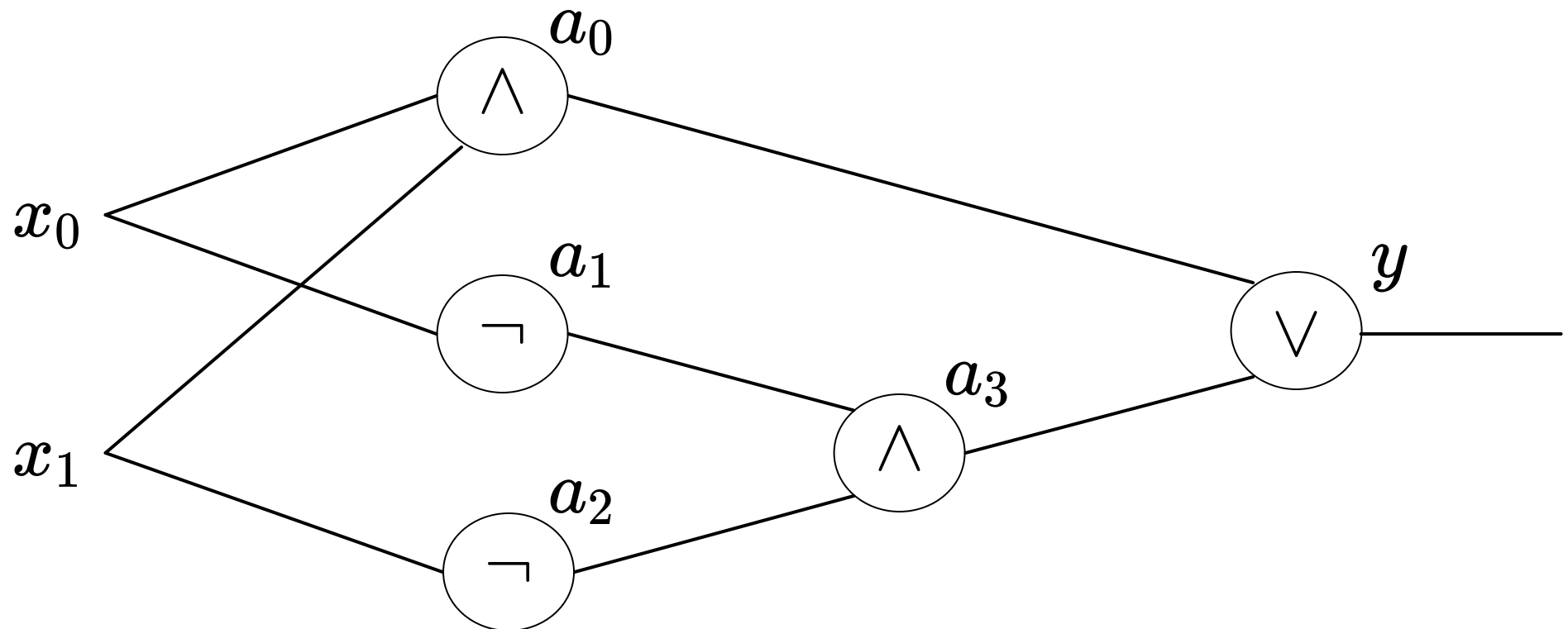
Construcción reversible a partir de un circuito clásico

Considere este ejemplo de circuito clásico que vimos en la introducción:

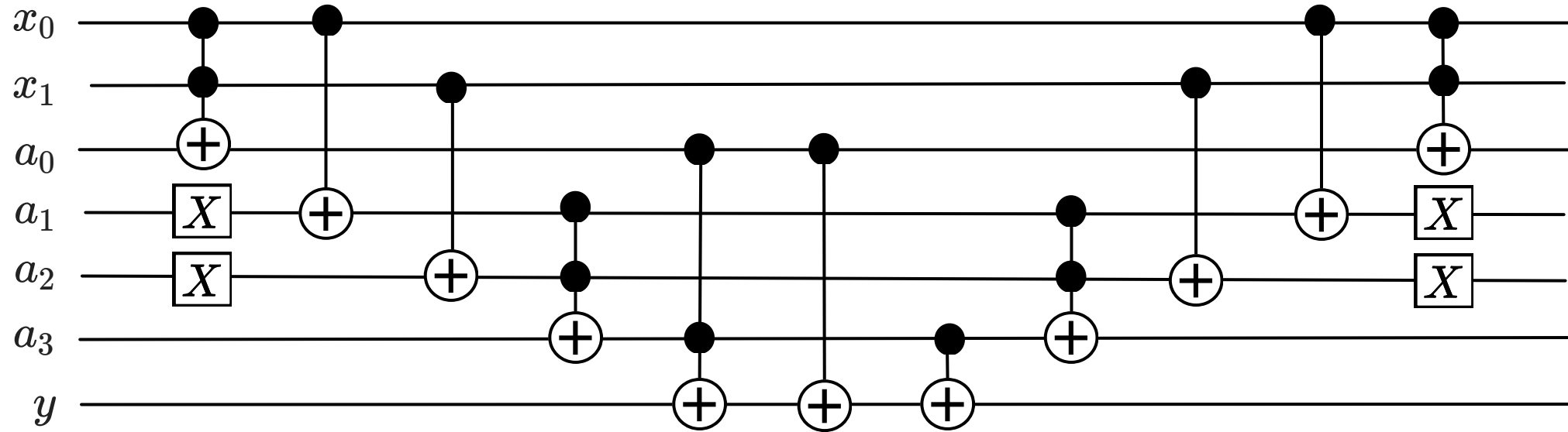


Construcción reversible a partir de un circuito clásico

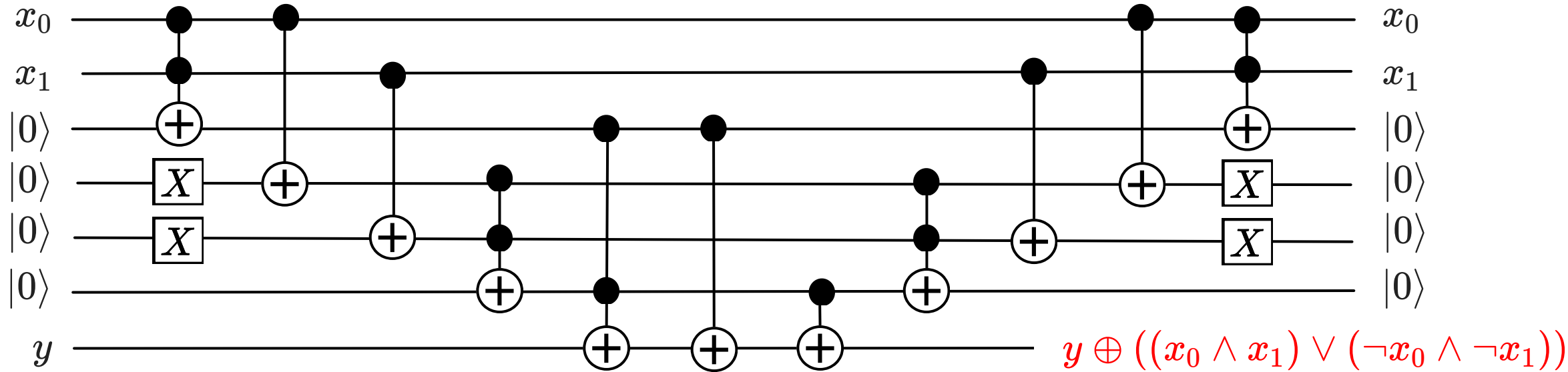
Considere este ejemplo de circuito clásico que vimos en la introducción:



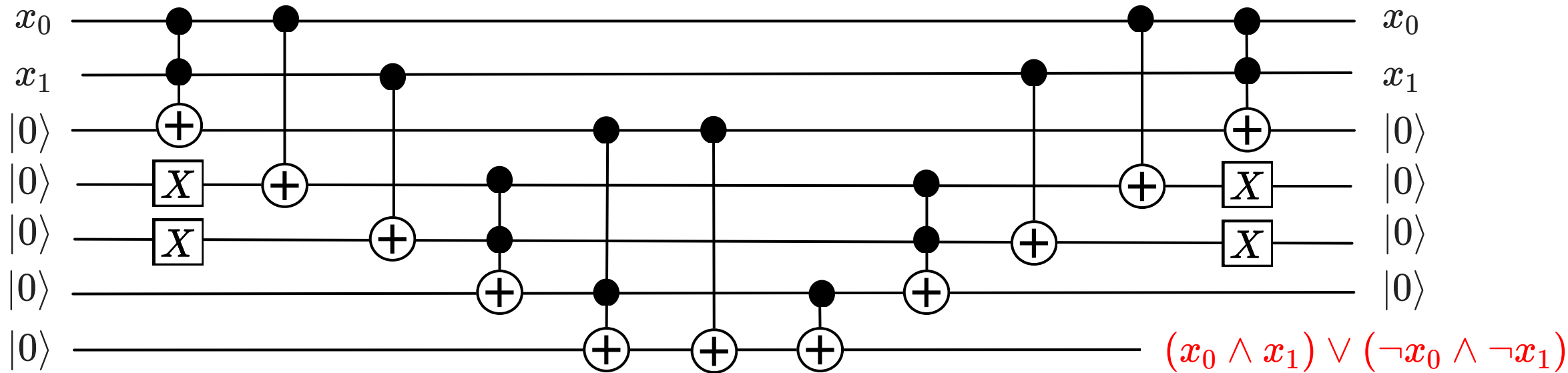
Construcción reversible a partir de un circuito clásico



Construcción reversible a partir de un circuito clásico



Construcción reversible a partir de un circuito clásico



Construcción reversible a partir de un circuito clásico

A partir de un circuito clásico C_f para $f : \{0, 1\}^n \rightarrow \{0, 1\}$, podemos construir un circuito que implementa la transformación unitaria U_f :

$$U_f |y 0^r x\rangle = |(y \oplus f(x)) 0^r x\rangle$$

Construcción reversible a partir de un circuito clásico

Suponga que C_f tiene S compuertas con a lo más dos entradas cada una

- La compuerta $\neg$ tiene una entrada
- Las compuertas $\wedge$ y $\vee$ tienen dos entradas cada una

La construcción reversible utiliza $O(S)$ compuertas y $O(S)$ qubits auxiliares

El problema de Deutsch

Consideremos una función $f : \{0, 1\} \rightarrow \{0, 1\}$

Queremos determinar si la función f es constante o balanceada

- Constante: $f(0) = f(1)$
- Balanceada: $f(0) \neq f(1)$

De manera equivalente, queremos calcular $f(0) \oplus f(1)$

Complejidad clásica del problema de Deutsch

¿Cuántas preguntas sobre f tenemos que hacer en un computador clásico para determinar si es constante o balanceada?

No nos basta sólo con conocer $f(0)$, ni tampoco con conocer $f(1)$

- Necesitamos realizar dos consultas

Complejidad cuántica del problema de Deutsch

¿Cuántos cálculos con U_f tenemos que hacer en un circuito cuántico para determinar si f es constante o balanceada?

El algoritmo de Deutsch muestra que basta con un solo acceso

El oráculo U_f

El oráculo U_f realiza el siguiente cálculo, suponiendo que $x, y \in \{0, 1\}$:

$$U_f|yx\rangle = |(y \oplus f(x))x\rangle$$

Accedemos al oráculo como una caja negra

- De todas formas es útil ver cómo se pueden implementar estos oráculos

El oráculo U_f

Casos constantes:

$$f(0) = f(1) = 0:$$

$$f(0) = f(1) = 1:$$

El oráculo U_f

Casos constantes:

$$f(0) = f(1) = 0:$$

x _____
 y _____

$$f(0) = f(1) = 1:$$

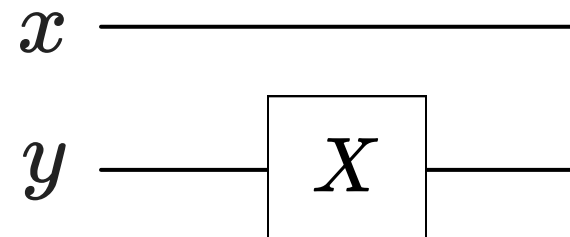
El oráculo U_f

Casos constantes:

$$f(0) = f(1) = 0:$$



$$f(0) = f(1) = 1:$$



El oráculo U_f

Casos balanceados

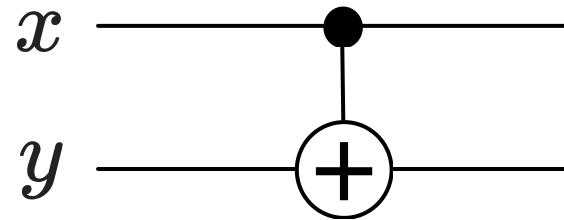
$f(0) = 0$ y $f(1) = 1$:

$f(0) = 1$ y $f(1) = 0$:

El oráculo U_f

Casos balanceados

$f(0) = 0$ y $f(1) = 1$:

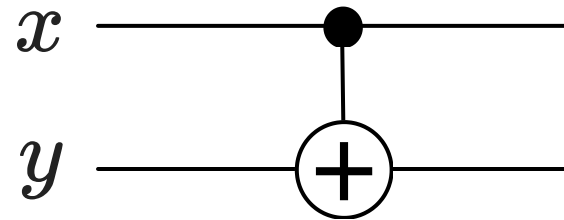


$f(0) = 1$ y $f(1) = 0$:

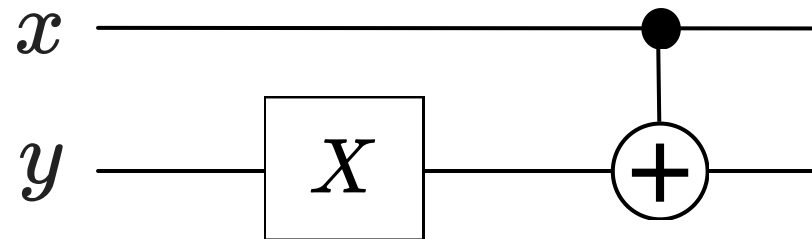
El oráculo U_f

Casos balanceados

$f(0) = 0$ y $f(1) = 1$:

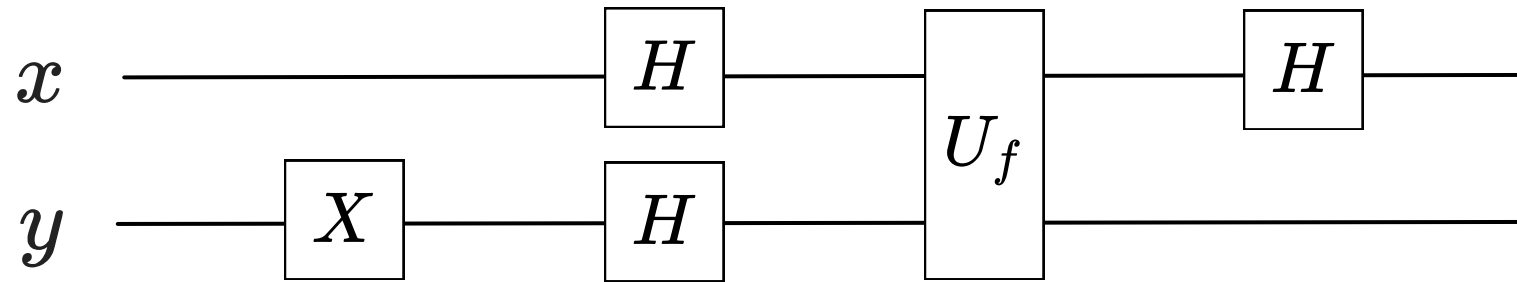


$f(0) = 1$ y $f(1) = 0$:



El circuito de Deutsch

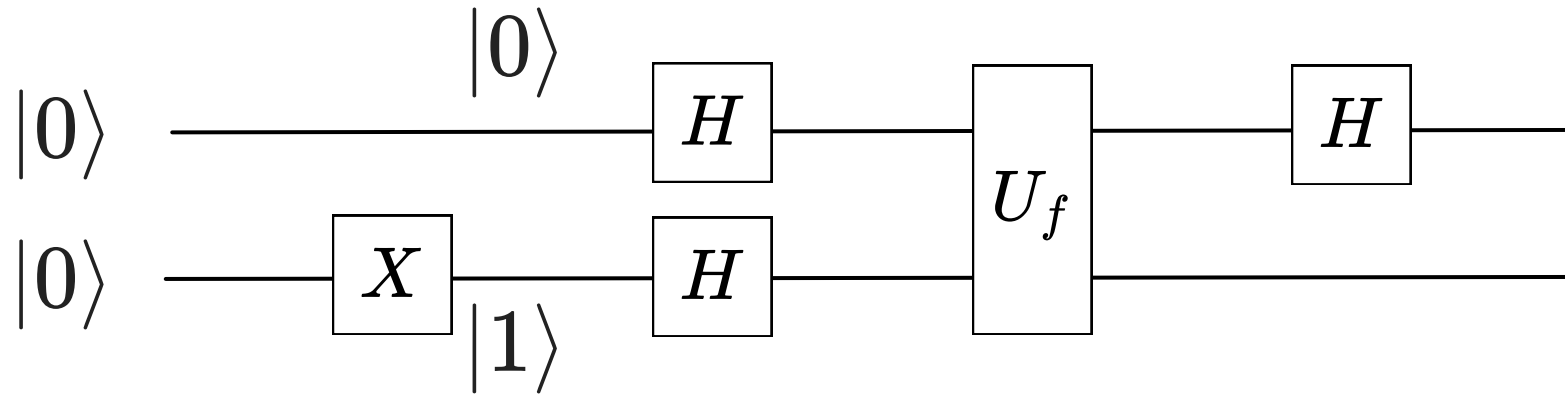
El siguiente circuito resuelve el problema:



La siguiente es la matriz que representa al circuito:

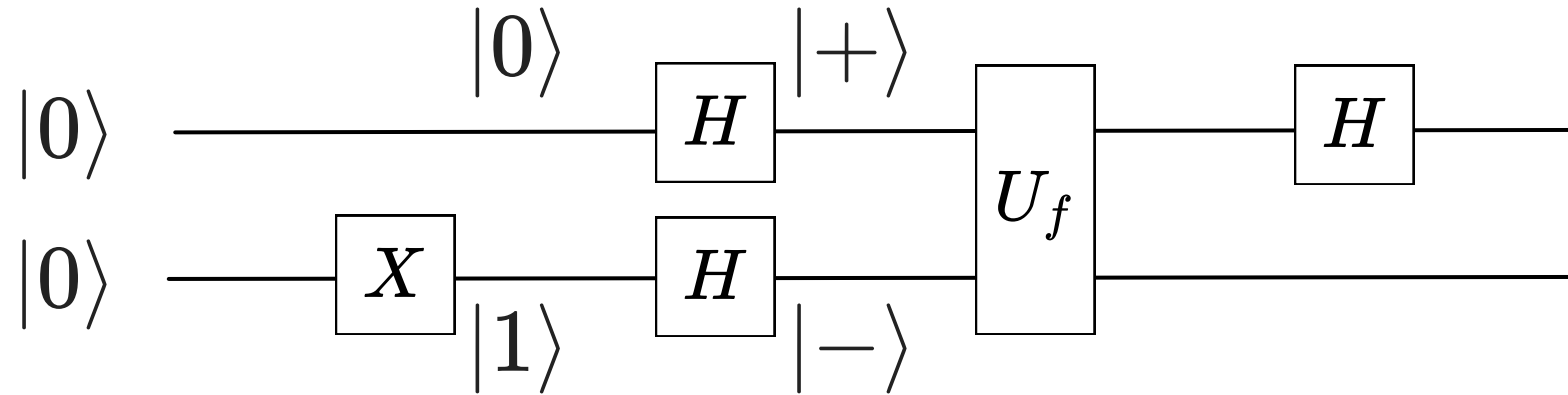
$$D_f = (I \otimes H)U_f(H \otimes H)(X \otimes I)$$

El circuito de Deutsch



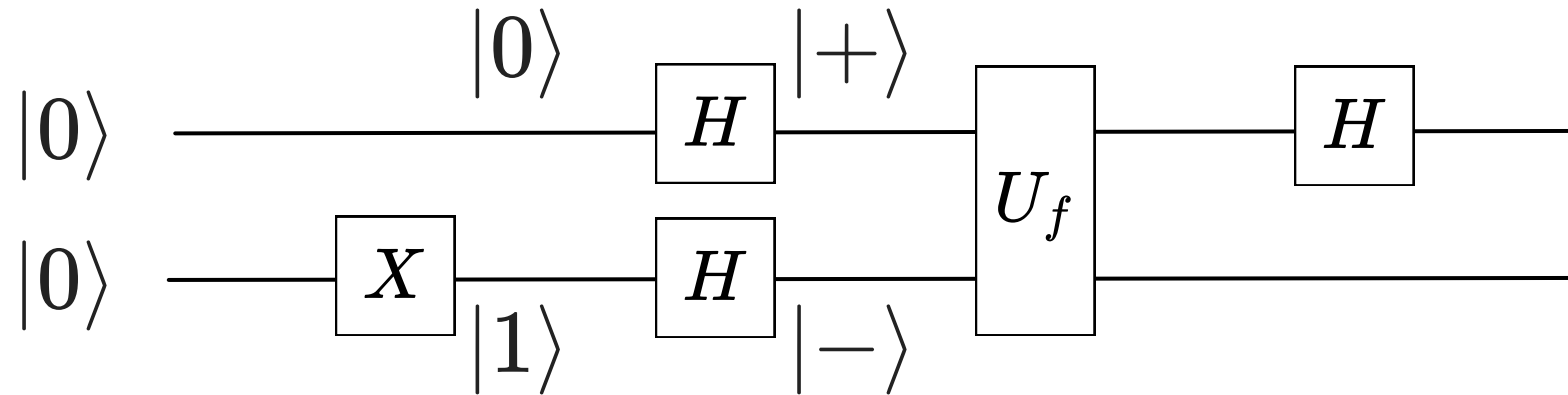
$$(X \otimes I)(|0\rangle \otimes |0\rangle) = |1\rangle \otimes |0\rangle$$

El circuito de Deutsch



$$(H \otimes H)(|1\rangle \otimes |0\rangle) = |-\rangle \otimes |+\rangle$$

El circuito de Deutsch



$$U_f(|-\rangle \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}})$$

El circuito de Deutsch

Suponga que $x \in \{0, 1\}^n$

Tenemos que:

$$\begin{aligned} U_f (|-\rangle \otimes |x\rangle) &= U_f \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \otimes |x\rangle \right) = U_f \left(\frac{|0x\rangle - |1x\rangle}{\sqrt{2}} \right) \\ &= \frac{U_f |0x\rangle - U_f |1x\rangle}{\sqrt{2}} = \frac{|f(x)x\rangle - |(1 \oplus f(x))x\rangle}{\sqrt{2}} = \frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} \otimes |x\rangle \end{aligned}$$

El circuito de Deutsch

Distinguimos dos casos:

$$\frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} = \begin{cases} |-\rangle & f(x) = 0 \\ -|-\rangle & f(x) = 1 \end{cases}$$

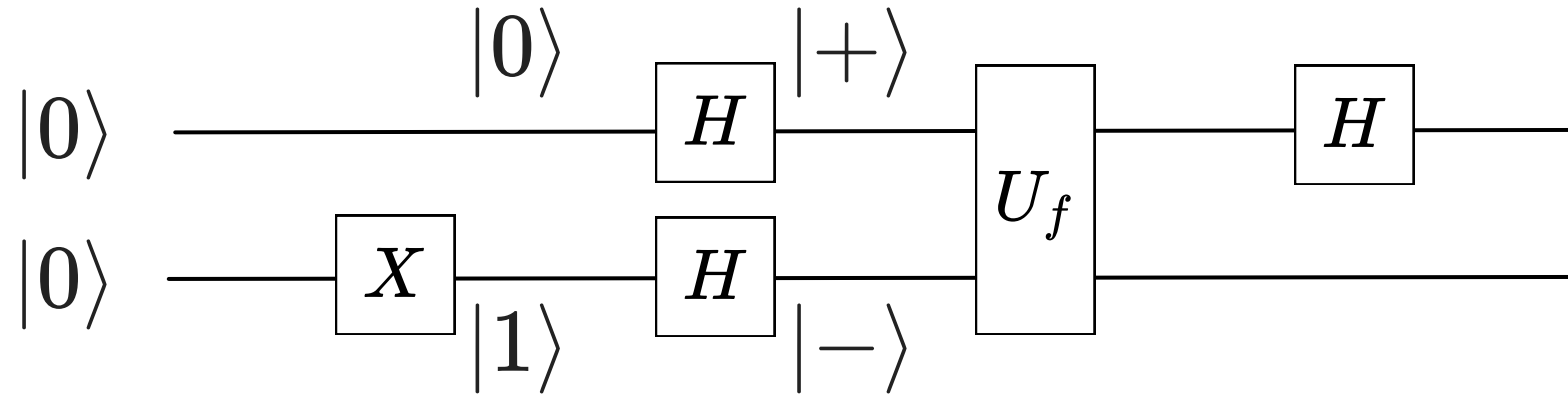
Por lo tanto: $U_f(|-\rangle \otimes |x\rangle) = ((-1)^{f(x)}|-\rangle) \otimes |x\rangle = |-\rangle \otimes (-1)^{f(x)}|x\rangle$

El circuito de Deutsch

Por linealidad:

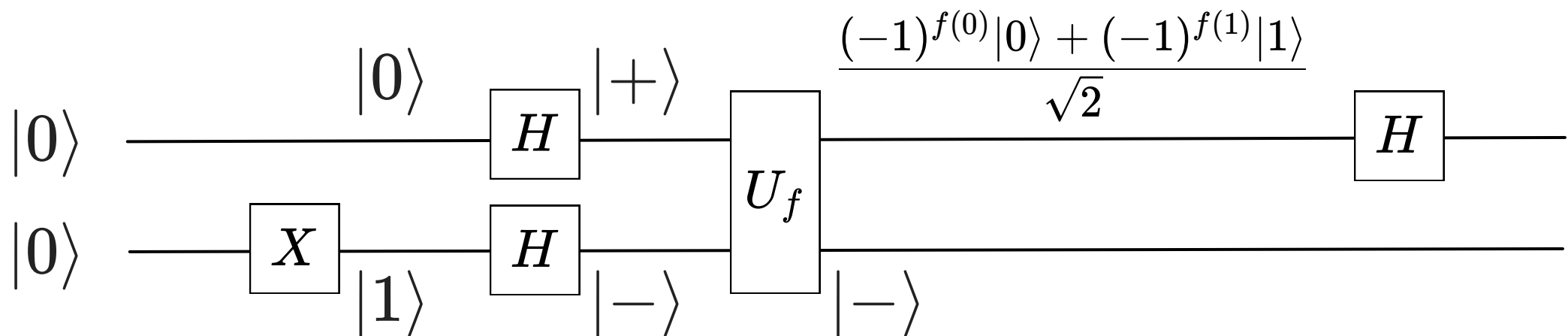
$$\begin{aligned} U_f \left(|-\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) &= U_f \left(\sum_{x \in \{0,1\}^n} \alpha_x (|-\rangle \otimes |x\rangle) \right) \\ &= \sum_{x \in \{0,1\}^n} \alpha_x U_f (|-\rangle \otimes |x\rangle) \\ &= \sum_{x \in \{0,1\}^n} \alpha_x (|-\rangle \otimes (-1)^{f(x)} |x\rangle) \\ &= \sum_{x \in \{0,1\}^n} |-\rangle \otimes \alpha_x (-1)^{f(x)} |x\rangle \\ &= |-\rangle \otimes \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle \end{aligned}$$

El circuito de Deutsch



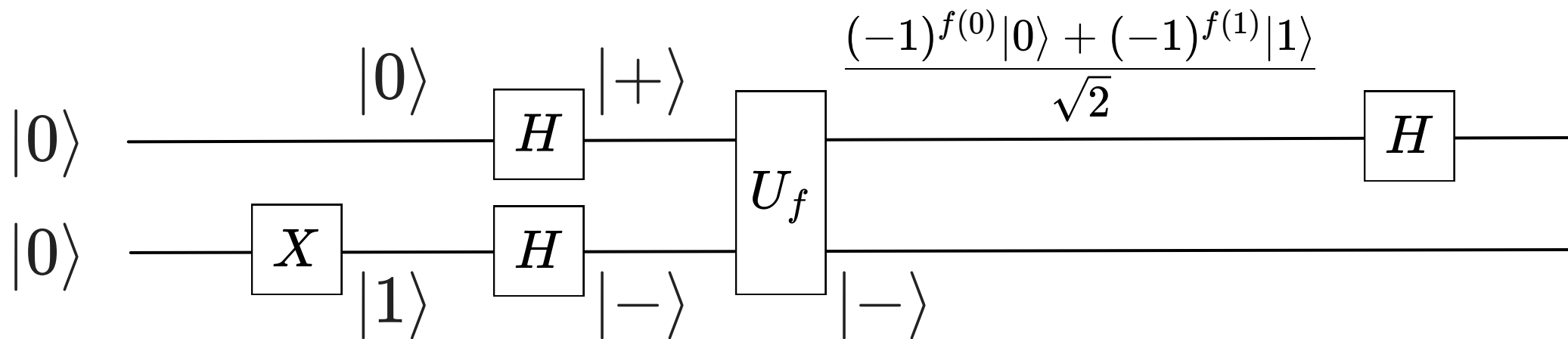
Concluimos que:
$$U_f(|-\rangle \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}}) = |-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}$$

El circuito de Deutsch



$$U_f(|-\rangle \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}}) = |-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}$$

El circuito de Deutsch



$$(I \otimes H) \left(|-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right)$$

El circuito de Deutsch

$$\begin{aligned} & (I \otimes H) \left(|-\rangle \otimes \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \\ &= I|-\rangle \otimes H \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \\ &= |-\rangle \otimes \left(\frac{(-1)^{f(0)}}{\sqrt{2}} H|0\rangle + \frac{(-1)^{f(1)}}{\sqrt{2}} H|1\rangle \right) \\ &= |-\rangle \otimes \left(\frac{(-1)^{f(0)}}{\sqrt{2}} \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) + \frac{(-1)^{f(1)}}{\sqrt{2}} \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \right) \\ &= |-\rangle \otimes \left(\frac{(-1)^{f(0)} + (-1)^{f(1)}}{2} |0\rangle + \frac{(-1)^{f(0)} - (-1)^{f(1)}}{2} |1\rangle \right) \end{aligned}$$

interferencia

El circuito de Deutsch

Por lo tanto, el siguiente es el resultado del circuito:

$$D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes \left(\frac{(-1)^{f(0)} + (-1)^{f(1)}}{2} |0\rangle + \frac{(-1)^{f(0)} - (-1)^{f(1)}}{2} |1\rangle \right)$$

El resultado depende de si f es constante o balanceada:

$$f \text{ es constante} \quad \longrightarrow \quad D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |0\rangle$$

$$f \text{ es balanceada} \quad \longrightarrow \quad D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |1\rangle$$

El circuito de Deutsch

$$f \text{ es constante} \longrightarrow D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |0\rangle$$

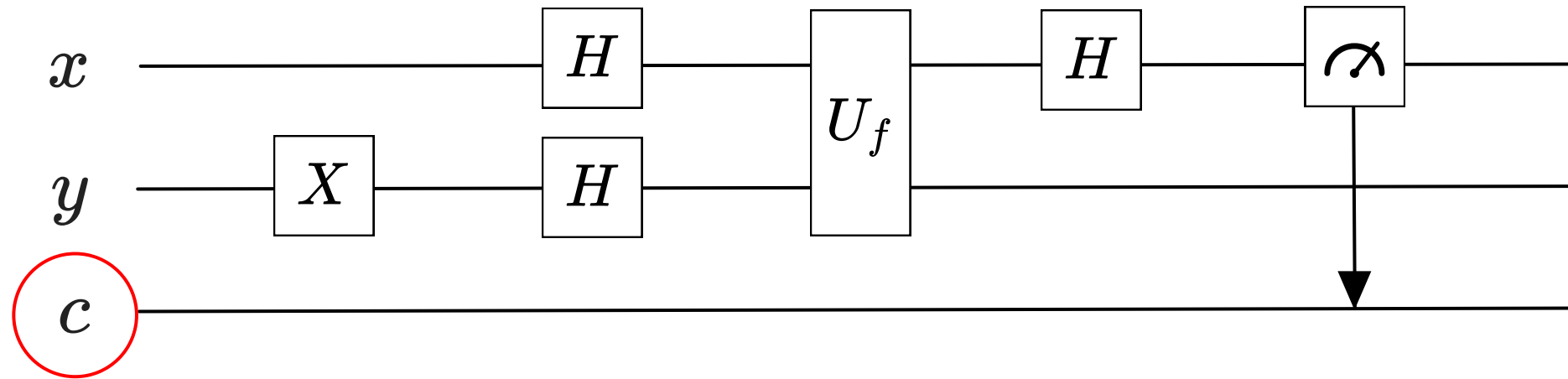
$$f \text{ es balanceada} \longrightarrow D_f(|0\rangle \otimes |0\rangle) = |-\rangle \otimes (-1)^{f(0)} |1\rangle$$

Midiendo el qubit menos significativo podemos saber si f es constante o balanceada

- Solo usamos el oráculo U_f una vez en el cálculo de D_f

El circuito de Deutsch

El siguiente es el circuito completo que resuelve el problema:



bit clásico donde se almacena
el resultado de la medición

El circuito de Deutsch en Qiskit

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

def oraculo(v0, v1):
    circuito = QuantumCircuit(2)

    if v0 == 0:
        if v1 == 1:
            circuito.cx(0, 1)
    else:
        if v1 == 0:
            circuito.x(1)
            circuito.cx(0, 1)
        else:
            circuito.x(1)

    return circuito
```

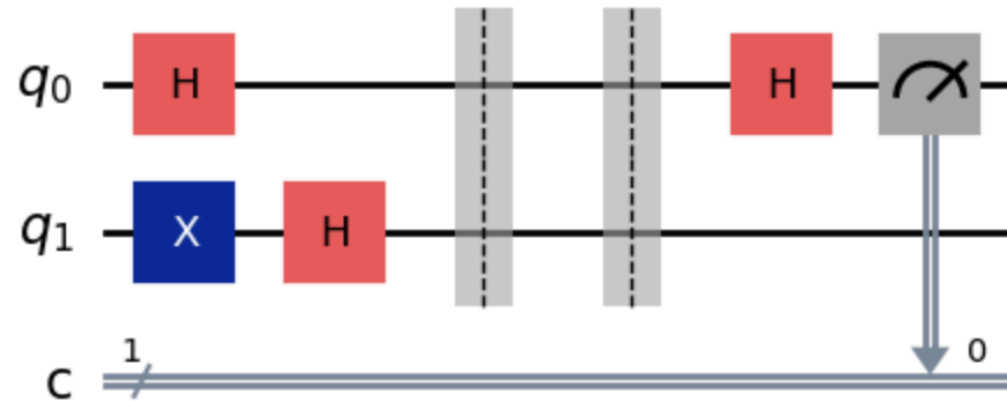
El circuito de Deutsch en Qiskit

```
def circuito_deutsch(oraculo):  
    circuito = QuantumCircuit(2, 1)  
  
    circuito.x(1)  
    circuito.h(0)  
    circuito.h(1)  
    circuito.barrier()  
    circuito.compose(oraculo, inplace=True)  
    circuito.barrier()  
    circuito.h(0)  
    circuito.measure(0, 0)  
  
    return circuito
```

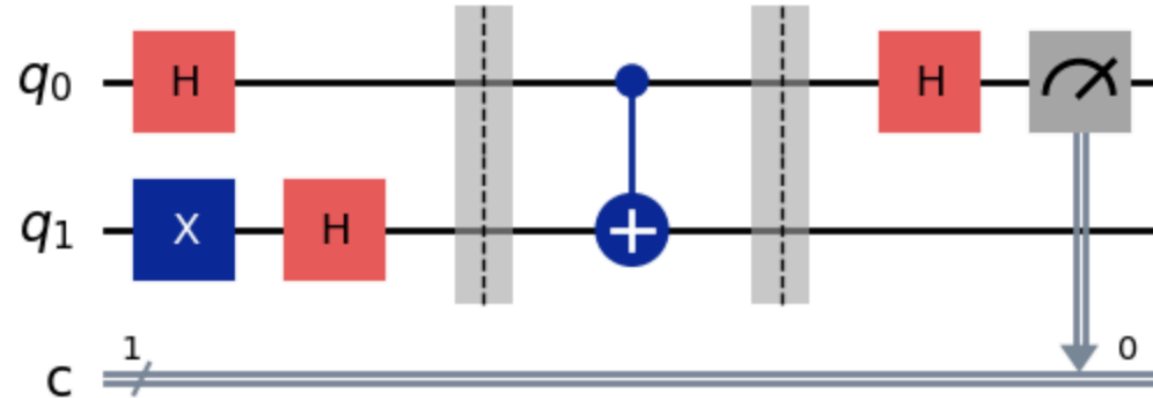
El circuito de Deutsch en Qiskit

```
for v0, v1 in [(0,0), (0,1), (1,0), (1,1)]:  
    U_f = oraculo(v0, v1)  
    D_f = circuito_deutsch(U_f)  
  
    simulador = StatevectorSampler()  
    resultado = simulador.run(  
        [D_f],  
        shots=1000  
    ).result()  
  
    display(D_f.draw("mpl"))  
    conteos = resultado[0].data.c.get_counts()  
    print(conteos)
```

El circuito de Deutsch en Qiskit

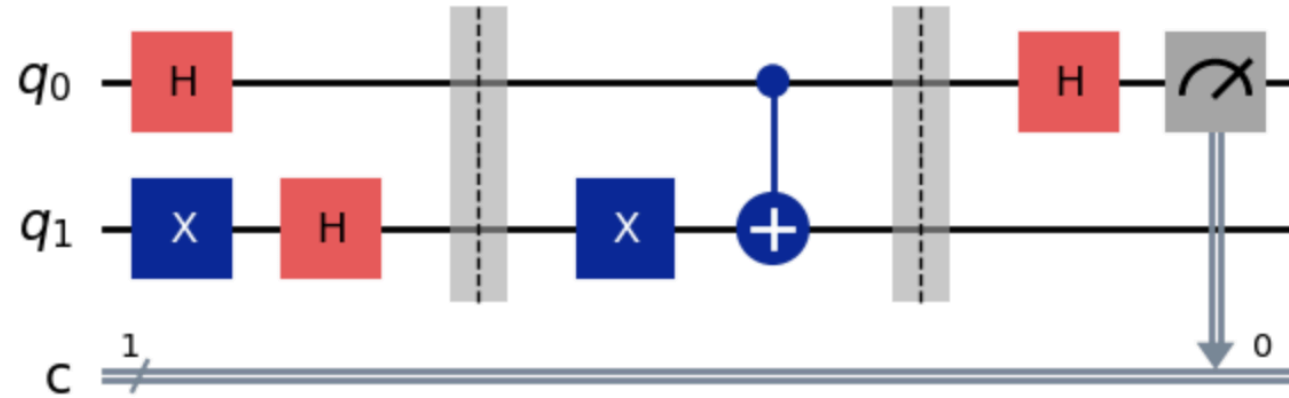


`{'0': 1000}`

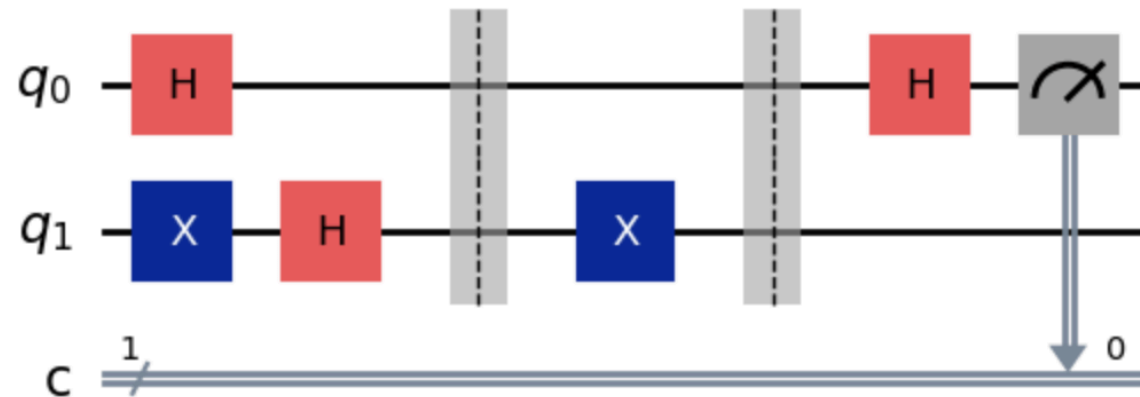


`{'1': 1000}`

El circuito de Deutsch en Qiskit



`{'1': 1000}`



`{'0': 1000}`

El problema de Deutsch–Jozsa

Consideremos ahora una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$

Tenemos la promesa de que f es constante o balanceada

- Constante: $f(x)$ tiene el mismo valor para todo $x \in \{0, 1\}^n$
- Balanceada: $|\{x \in \{0, 1\}^n \mid f(x) = 0\}| = 2^{n-1}$

El objetivo es determinar cuál de los dos casos ocurre

- Para $n = 1$ obtenemos el problema de Deutsch

Complejidad clásica del problema de Deutsch-Jozsa

¿Cuántas preguntas sobre f tenemos que hacer en un computador clásico para determinar si es constante o balanceada?

En el peor caso, necesitamos realizar al menos $2^{n-1} + 1$ consultas si vamos a utilizar un algoritmo determinista

¿Cuántas consultas sobre f se deben hacer si se permite un algoritmo aleatorizado?

- ¿Cuál es el error de este algoritmo?

Complejidad cuántica del problema de Deutsch–Jozsa

¿Cuántos cálculos con U_f tenemos que hacer en un circuito cuántico para determinar si f es constante o balanceada?

El algoritmo de Deutsch–Jozsa muestra que basta con un solo acceso

- Este es un algoritmo determinista

Deutsch–Jozsa tiene una ventaja exponencial en el número de consultas frente a los algoritmos clásicos deterministas

El oráculo U_f

Suponemos que el oráculo U_f realiza el siguiente cálculo, dado $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

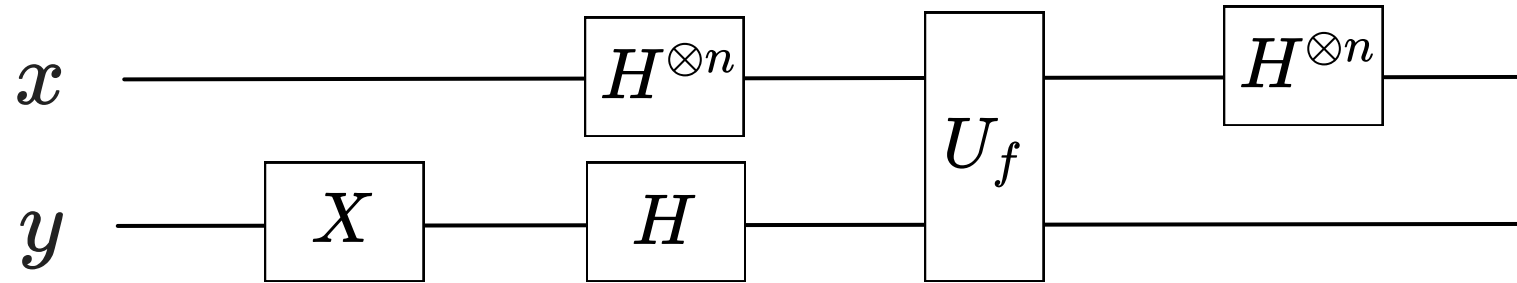
$$U_f|yx\rangle = |(y \oplus f(x))x\rangle$$

Más adelante vamos a discutir cómo se implementa el algoritmo considerando un oráculo U_f con qubits auxiliares:

$$U_f|y0^r x\rangle = |(y \oplus f(x))0^r x\rangle$$

El circuito de Deutsch-Jozsa

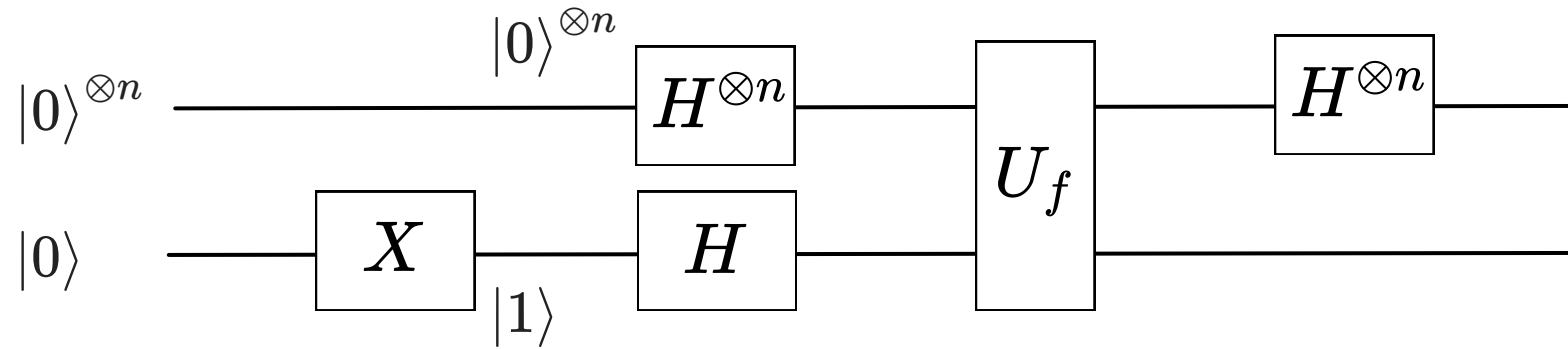
El siguiente circuito resuelve el problema:



La siguiente es la matriz que representa al circuito:

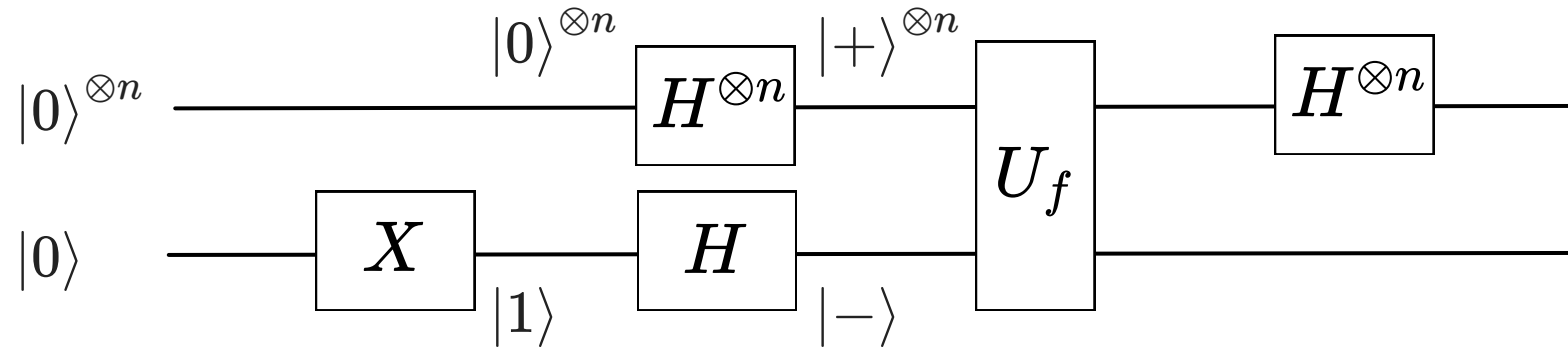
$$D_f = (I \otimes H^{\otimes n})U_f(H \otimes H^{\otimes n})(X \otimes I^{\otimes n})$$

El circuito de Deutsch-Jozsa



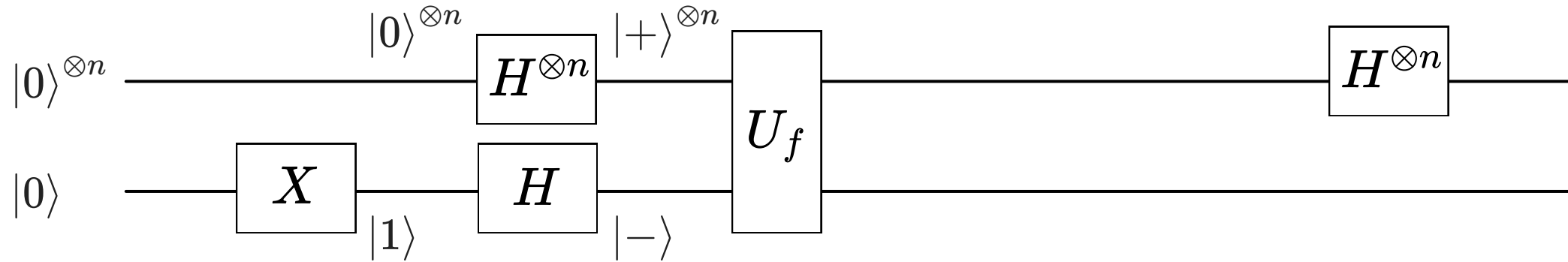
$$(X \otimes I^{\otimes n})(|0\rangle \otimes |0\rangle^{\otimes n}) = |1\rangle \otimes |0\rangle^{\otimes n}$$

El circuito de Deutsch-Jozsa



$$(H \otimes H^{\otimes n})(|1\rangle \otimes |0\rangle^{\otimes n}) = |-\rangle \otimes |+\rangle^{\otimes n}$$

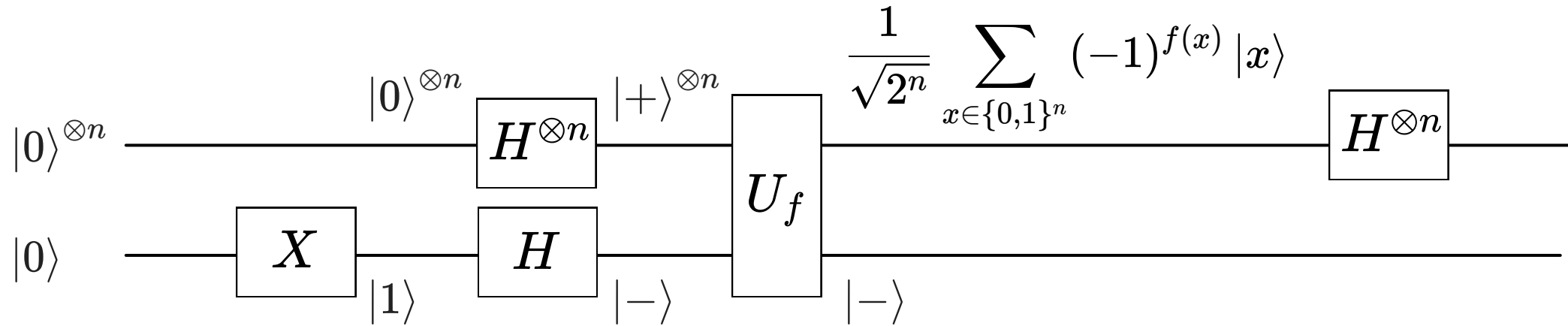
El circuito de Deutsch-Jozsa



$$U_f(|-\rangle \otimes |+\rangle^{\otimes n})$$

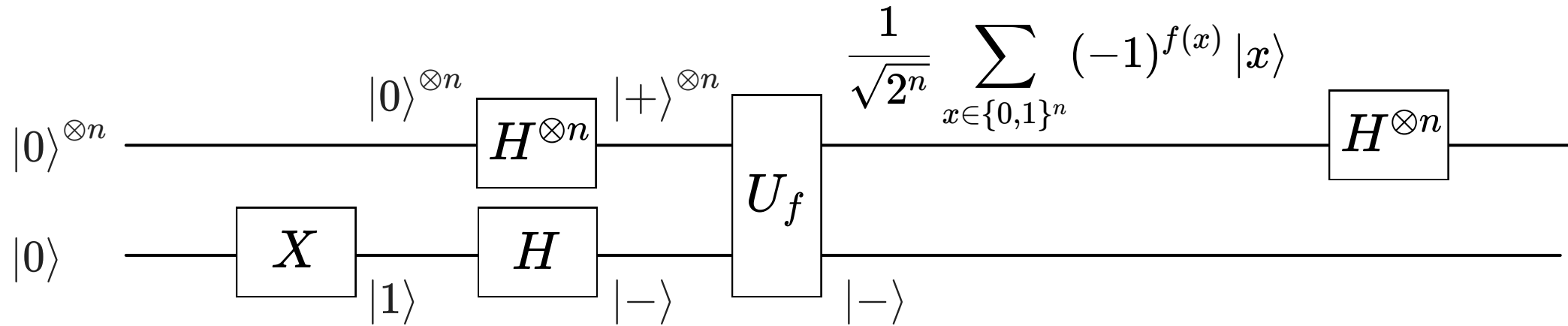
Tenemos que:
$$|+\rangle^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

El circuito de Deutsch-Jozsa



$$U_f \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \right) = |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle$$

El circuito de Deutsch-Jozsa



$$(I \otimes H^{\otimes n}) \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right)$$

El circuito de Deutsch-Jozsa

$$\begin{aligned} (I \otimes H^{\otimes n}) & \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right) \\ &= I |-\rangle \otimes H^{\otimes n} \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \\ &= |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle \end{aligned}$$

Tenemos que calcular entonces $H^{\otimes n} |x\rangle$ para $x \in \{0,1\}^n$

El circuito de Deutsch-Jozsa

Suponga que $x = x_{n-1} \cdots x_0$, donde $x_i \in \{0, 1\}$ para cada $i \in \{0, \dots, n-1\}$

Tenemos que:

$$\begin{aligned} H^{\otimes n} |x\rangle &= H^{\otimes n} (|x_{n-1}\rangle \otimes \cdots \otimes |x_0\rangle) \\ &= \bigotimes_{i=1}^n H |x_{n-i}\rangle = \bigotimes_{i=1}^n \frac{|0\rangle + (-1)^{x_{n-i}} |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{z \cdot x} |z\rangle \end{aligned}$$

$$\text{donde } z \cdot x = \bigoplus_{i=0}^{n-1} z_i x_i, \text{ para } z = z_{n-1} \cdots z_0$$

El circuito de Deutsch-Jozsa

Por lo tanto:

$$\begin{aligned} & (I \otimes H^{\otimes n}) \left(|-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \right) \\ &= |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle \\ &= |-\rangle \otimes \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \left(\frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{z \cdot x} |z\rangle \right) \\ &= |-\rangle \otimes \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{z \in \{0,1\}^n} (-1)^{f(x)} (-1)^{z \cdot x} |z\rangle \\ &= |-\rangle \otimes \frac{1}{2^n} \sum_{z \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{f(x) + z \cdot x} \right) |z\rangle \end{aligned}$$

El circuito de Deutsch-Jozsa

Tenemos entonces que si medimos los n qubits menos significativos, la probabilidad de obtener $|0^n\rangle$ es:

$$p = \left(\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \right)^2$$

Esta probabilidad depende de si f es constante o balanceada:

$$f \text{ es constante} \longrightarrow p = 1$$

$$f \text{ es balanceada} \longrightarrow p = 0$$

El circuito de Deutsch-Jozsa

f es constante $\longrightarrow p = 1$

f es balanceada $\longrightarrow p = 0$

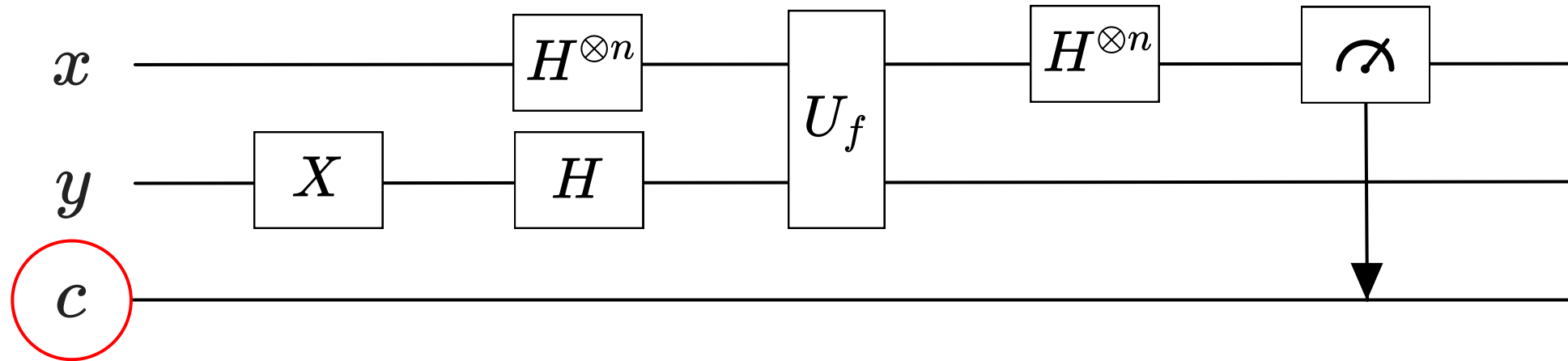
Midiendo los n qubits menos significativos podemos saber si f es constante o balanceada

- Si el resultado es $|0^n\rangle$, entonces f es constante
- Si el resultado es distinto de $|0^n\rangle$, entonces f es balanceada

Solo usamos el oráculo U_f una vez en el cálculo de D_f

El circuito de Deutsch-Jozsa

El siguiente es el circuito completo que resuelve el problema:



n bits clásicos donde se almacena
el resultado de la medición

El circuito de Deutsch–Jozsa

Ejercicio: Resuelva el problema de Deutsch–Jozsa suponiendo que tiene un oráculo U_f para f con qubits auxiliares:

$$U_f|y0^r x\rangle = |(y \oplus f(x))0^r x\rangle$$

- El circuito es esencialmente el mismo que el anterior

El oráculo de fase

Para una función Booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$, el **oráculo de fase** O_f se define de la siguiente forma, suponiendo que $x \in \{0, 1\}^n$:

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle$$

Por lo tanto:

$$O_f \left(\sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) = \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle$$

Las entradas para las cuales $f(x) = 1$ reciben un signo negativo

¿Existe el oráculo de fase para cualquier función Booleana?

Sabemos que O_f es una matriz de $2^n \times 2^n$

Vamos a demostrar que O_f es una matriz unitaria

- Para esto, vamos a demostrar primero que $O_f O_f = I$, donde I es la matriz identidad de $2^n \times 2^n$

Consideramos un vector arbitrario $|\psi\rangle \in \mathbb{C}^{2^n}$. Vamos a demostrar que $O_f O_f |\psi\rangle = |\psi\rangle$

- De esto se concluye que $O_f O_f = I$

¿Existe el oráculo de fase para cualquier función Booleana?

Sabemos que: $|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$

Por lo tanto:

$$\begin{aligned} O_f O_f |\psi\rangle &= O_f O_f \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \\ &= O_f \sum_{x \in \{0,1\}^n} \alpha_x (-1)^{f(x)} |x\rangle \\ &= \sum_{x \in \{0,1\}^n} \alpha_x (-1)^{f(x)} (-1)^{f(x)} |x\rangle \\ &= \sum_{x \in \{0,1\}^n} (-1)^{2f(x)} \alpha_x |x\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \end{aligned}$$

¿Existe el oráculo de fase para cualquier función Booleana?

Tenemos que: $O_f O_f = I$

Vamos a demostrar que $O_f^\dagger = O_f$

- De esto concluimos que O_f es una matriz unitaria puesto que $O_f^\dagger O_f = O_f O_f = I$

¿Existe el oráculo de fase para cualquier función Booleana?

Considere la matriz: $\sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|$

Tenemos lo siguiente para $x \in \{0,1\}^n$:

$$\begin{aligned} \left(\sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u| \right) |x\rangle &= \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|x\rangle \\ &= \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|x\rangle = (-1)^{f(x)} |x\rangle = O_f |x\rangle \end{aligned}$$

¿Existe el oráculo de fase para cualquier función Booleana?

Concluimos que: $O_f = \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u|$

Por lo tanto:

$$\begin{aligned} O_f^\dagger &= \left(\sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u| \right)^\dagger \\ &= \sum_{u \in \{0,1\}^n} ((-1)^{f(u)} |u\rangle \langle u|)^\dagger \\ &= \sum_{u \in \{0,1\}^n} \overline{(-1)^{f(u)}} (|u\rangle \langle u|)^\dagger = \sum_{u \in \{0,1\}^n} (-1)^{f(u)} |u\rangle \langle u| = O_f \end{aligned}$$

¿Existe el oráculo de fase para cualquier función Booleana?

Tenemos entonces que la matriz O_f es unitaria

Concluimos que toda función booleana $f : \{0, 1\}^n \rightarrow \{0, 1\}$ admite un oráculo de fase unitario

- f admite un oráculo que puede ser implementado
- ¿Cómo construimos este operador usando las compuertas que conocemos?

De una respuesta a una fase

Vamos a ver cómo obtener O_f a partir de U_f

Sabemos que:

$$U_f \left(|-\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) = |-\rangle \otimes \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle$$

De una respuesta a una fase

Por lo tanto, extendiendo O_f con un qubit auxiliar:

$$O_f \left(|0\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) = |0\rangle \otimes \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle$$

Tenemos que:

$$O_f = (X \otimes I^{\otimes n})(H \otimes I^{\otimes n})U_f(H \otimes I^{\otimes n})(X \otimes I^{\otimes n})$$

De una respuesta a una fase

En general tenemos que $U_f(|y\rangle \otimes |0^r\rangle \otimes |x\rangle) = |y \oplus f(x)\rangle \otimes |0^r\rangle \otimes |x\rangle$

- Usamos r qubits auxiliares en la implementación de U_f

Como en el caso anterior construimos O_f utilizando U_f :

$$O_f = (X \otimes I^{\otimes(r+n)})(H \otimes I^{\otimes(r+n)})U_f(H \otimes I^{\otimes(r+n)})(X \otimes I^{\otimes(r+n)})$$

De una respuesta a una fase

Como en el caso anterior construimos O_f utilizando U_f :

$$O_f = (X \otimes I^{\otimes(r+n)})(H \otimes I^{\otimes(r+n)})U_f(H \otimes I^{\otimes(r+n)})(X \otimes I^{\otimes(r+n)})$$

En este caso tenemos que:

$$O_f \left(|0^{1+r}\rangle \otimes \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \right) = |0^{1+r}\rangle \otimes \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \alpha_x |x\rangle$$

Esta implementación más general de O_f usa $r + 1$ qubits auxiliares

¿Es posible definir U_f a partir de O_f ?

Vamos a mostrar que esto no es posible

- Incluso para funciones muy simples

Sean $f : \{0, 1\} \rightarrow \{0, 1\}$ y $g : \{0, 1\} \rightarrow \{0, 1\}$ las siguientes funciones constantes:

$$f(0) = f(1) = 0$$

$$g(0) = g(1) = 1$$

¿Es posible definir U_f a partir de O_f ?

Sean $f : \{0, 1\} \rightarrow \{0, 1\}$ y $g : \{0, 1\} \rightarrow \{0, 1\}$ las siguientes funciones constantes:

$$f(0) = f(1) = 0$$

$$g(0) = g(1) = 1$$

Para $|\psi\rangle \in \mathbb{C}^2$, tenemos que: $O_f|\psi\rangle = |\psi\rangle$ y $O_g|\psi\rangle = -|\psi\rangle$

- El resultado de los oráculos es el mismo excepto por una fase global

¿Es posible definir U_f a partir de O_f ?

Por otro lado tenemos que $U_f|00\rangle = |00\rangle$ y $U_g|00\rangle = |10\rangle$

Para $|\psi\rangle \in \mathbb{C}^4$, puede suceder entonces que $U_f|\psi\rangle$ y $U_g|\psi\rangle$ no difieran solo en una fase global

- Por lo tanto, no hay una manera uniforme de expresar un oráculo XOR a partir de un oráculo de fase

¿Por qué utilizamos los oráculos de fase?

Como los oráculos de fase tienen menos información, en algunos casos pueden ser expresados de manera más simple (como un circuito cuántico) que los oráculos XOR

- En algunos algoritmos nos basta con utilizar oráculos de fase

Un ejemplo con oráculo de fase más simple

Por ejemplo, considere la función:

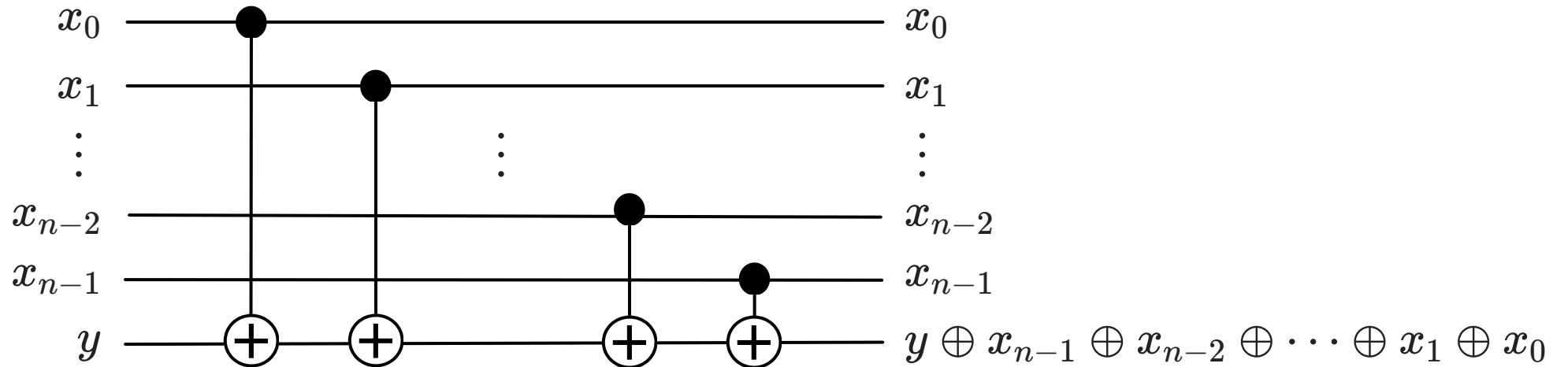
$$f(x_{n-1}, x_{n-2}, \dots, x_1, x_0) = x_{n-1} \oplus x_{n-2} \oplus \dots \oplus x_1 \oplus x_0$$

Un ejemplo con oráculo de fase más simple

Por ejemplo, considere la función:

$$f(x_{n-1}, x_{n-2}, \dots, x_1, x_0) = x_{n-1} \oplus x_{n-2} \oplus \dots \oplus x_1 \oplus x_0$$

El oráculo U_f :



Un ejemplo con oráculo de fase más simple

El oráculo U_f tiene n compuertas y toma tiempo n ejecutarlo

- El oráculo de fase implementado a partir de U_f toma tiempo $\Theta(n)$ en ser ejecutado

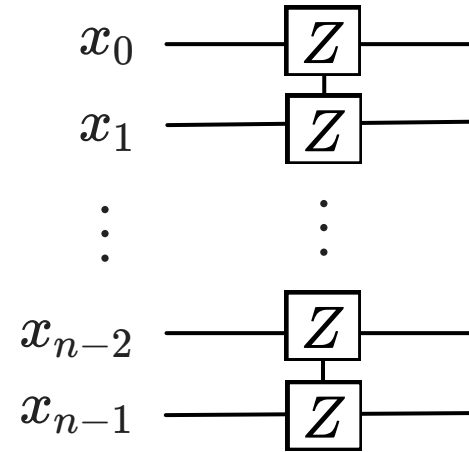
Pero para $x \in \{0, 1\}^n$, tenemos que:

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle = (-1)^{x_{n-1} \oplus x_{n-2} \oplus \dots \oplus x_1 \oplus x_0}|x\rangle$$

Por lo tanto: $O_f = Z^{\otimes n}$, donde $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Un ejemplo con oráculo de fase más simple

Construcción directa del oráculo O_f :



El oráculo O_f tiene n compuertas y toma tiempo 1 ejecutarlo

El circuito de Deutsch-Jozsa con oráculo de fase

En el algoritmo de Deutsch-Jozsa realmente estamos usando un oráculo de fase:

El circuito de Deutsch-Jozsa con oráculo de fase

En el algoritmo de Deutsch-Jozsa realmente estamos usando un oráculo de fase:

